Documentación APP Móvil

1. Introducción

En este documento se muestra la solución implementada para compilar aplicaciones .NET MAUI dirigidas a Android e IOS utilizando un único esquipo Windows, sin usar un equipo mac y sin pagar una licencia de Apple developer durante esta fase de pruebas

2. Proceso de investigación previa

Primero se investigaron los lenguajes de programación posibles para el desarrollo de aplicaciones móviles, como pueden ser React Native, Swift, Java, Kotlin y .NET MAUI. El lenguaje elegido fue .NET MAUI ya que es un lenguaje que utilizado durante mucho tiempo a lo que se suma su posibilidad de compilación en Android mediante la generación de un archivo apk y en IOS con archivo. ipa generado a partir de workflow en github

React Native y java fueron descartados por la dificultad principal que se ha encontrado durante el proceso, la necesidad de un equipo Mac para realizar la compilación y la firma de la aplicación y una cuenta de Apple Developer Program. Y Swift y Kotlin por la imposibilidad de hacer aplicaciones multiplataforma.

Como se ha mencionado antes párala versión final se ha usado .net Maui para el desarrollo, después mediante el workflow en Github se genera un archivo. ipa que mediante la aplicación Sideloadly se sube a tu cuenta de Apple ID y se puede usar en un dispositivo iPhone

3. Desarrollo de la app

La app se desarrolló en Visual Studio 2026, para el desarrollo multiplataforma instala la carga de trabajo llamada “Desarrollo de la interfaz de usuario de aplicaciones multiplataforma de .NET” después al crear el proyecto se selecciona “Aplicación .NET MAUI” lo que genera los archivos necesarios.

Para el front-end se usa el archivo MainPage.xaml que contiene el siguiente código que usa una sintaxis parecida a HTML:

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="pruebaMaui.MainPage">

    <ScrollView>
        <VerticalStackLayout Spacing="25" Padding="30">

            <Label Text="¡Bienvenido a MAUI!"
                   FontSize="32"
                   HorizontalOptions="Center" />

            <Button x:Name="CounterBtn"
                    Text="Haz clic aquí"
                    Clicked="OnCounterClicked"
                    HorizontalOptions="Center" />

        </VerticalStackLayout>
    </ScrollView>

</ContentPage>
```

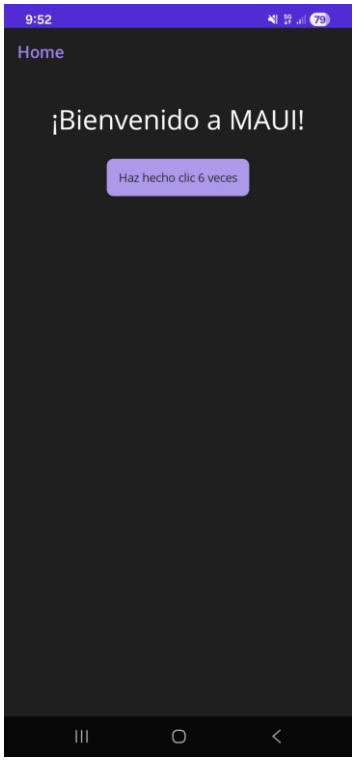
Para el back-end se usa el archivo MainPage.xaml.cs que usa C#:

```
namespace pruebaMaui;

15 referencias | Iker Benavente, Hace 18 horas | 1 autor, 1 cambio
public partial class MainPage : ContentPage
{
    int count = 0;

    0 referencias | Iker Benavente, Hace 18 horas | 1 autor, 1 cambio
    public MainPage()
    {
        InitializeComponent();
    }

    2 referencias | Iker Benavente, Hace 18 horas | 1 autor, 1 cambio
    private void OnCounterClicked(object sender, EventArgs e)
    {
        count++;
        CounterBtn.Text = $"Haz hecho clic {count} veces";
    }
}
```

4. Implementación en Android

Para la implementación en dispositivos Android se usa un archivo apk el cual se genera en la compilación del código con el siguiente comando **dotnet publish -f net10-android -c Release,** después de la compilación el archivo se encontrará en la siguiente ruta **bin/Release/net10-android/publish/com.companyname.pruebaMaui-Signed.apk**

Una vez generado, el archivo apk se puede enviar a cualquier dispositivo Android para instalar la aplicación y ejecutarla, después del proceso se inicia la app:



5. Implementación en IOS

Para la implementación en IOS se han encontrado algunos problemas por las restricciones del ecosistema de Apple, que requiere de un equipo Mac con Xcode para compilar y firmar el código. Para solucionar este problema se usa la compilación en la nube y firma local, que basta para realizar las pruebas necesarias antes de ensamblar y firmar la aplicación para subirla la app store para lo que si se necesita un equipo Mac. Esto supone mas de un 90% del desarrollo.

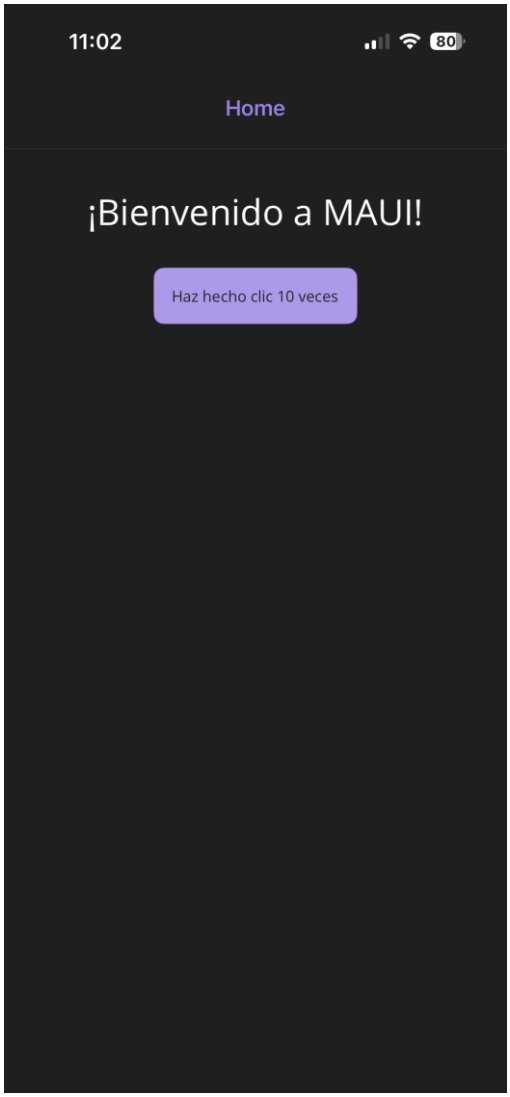
Para la compilación en la nube se usa Github actions que mediante un workflow creado en el repositorio genera el archivo necesario para su funcionamiento en un IPhone.

```yaml
name: Build iOS .ipa (para Sideloadly)

on:
  push:
    branches: [ main ]
  workflow_dispatch:

jobs:
  build-ios:
    runs-on: macos-26          # ← ESTO ES LO IMPORTANTE (cámbialo aquí)

    steps:
      - name: Checkout código
        uses: actions/checkout@v4

      - name: Setup .NET SDK
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '10.0.x'

      - name: Instalar workload MAUI
        run: dotnet workload install maui

      - name: Compilar y generar .ipa SIN firma
        run: |
          dotnet publish "${{ github.workspace }}/pruebaMaui.csproj" \
            --configuration Release \
            --framework net10.0-ios \
            -p:RuntimeIdentifier=ios-arm64 \
            -p:BuildIpa=true \
            -p:EnableCodeSigning=false \
            -p:EnablePackageSigning=false \
            -p:CodesignKey="" \
            -p:CodesignProvision="" \
            -p:SignBundle=false

      - name: Subir .ipa como artefacto
        uses: actions/upload-artifact@v4
        with:
          name: pruebaMaui-IPA
          path: '**/*.ipa'
          retention-days: 7
```

El wokflow funciona de la siguiente forma:

1. Inicia un Mac virtual en la nube
2. Descarga el código del repositorio
3. Instala las herramientas necesarias
4. Compila la app y genera un archivo .ipa
5. Devuelve un zip con el archivo .ipa

Una vez descomprimes la carpeta y obtienes el archivo lo subes a la aplicación Sideloadly mientras el Iphone esta conecta al PC mediante USB. Una vez se sube la aplicación te pide la cuenta de Apple ID, inicias sesión y se sube automáticamente al dispositivo



Aunque con este método se haya resuelto el problema de no usar el dispositivo Mac sigue habiendo algunos problemas. Por cuenta de Apple ID solo hay 3 huecos para subir aplicaciones, cada aplicación usa uno de los huecos que expira a los 7 días, para que no expire habría que tener una cuenta de Apple Developer Progrman la cual cuesta 99 al año

6. Conclusión

Con esta solución se permite probar las funcinalidades en IOS sin invertir en Hardware. Aunque esta solución signifique el 90% del proceso solo sirve para hacer pruebas de la app, si se quiere tener en producción se necesitaría un equipo Mac para firmar la aplicación y subirla a la app store